

PressurePause Reclaimer

PSI-Gated Direct Reclaim Coordination

CS325 Semester Project (CLO 2) — Linux Kernel Modification

Course: CS325 — Operating Systems
Instructor: Dr. Mohammad Imran
CLO: CLO 2
Reg. IDs: 241110, 242353
Kernel tree: Linux v6.8.12 (LOCALVERSION=-baseline)

Contents

1	Introduction	3
2	Environment	3
3	Kernel Build and Boot	3
3.1	Host Preparation	3
3.2	Source Tree	4
3.3	Kernel Configuration	5
3.4	Compilation	6
3.5	Installation	7
3.6	Bootloader Update	8
3.7	Boot Verification	9
4	Component Identification	10
4.1	Selected Component	10
4.2	Call Graph: Allocation to Reclaim	11
4.3	Key Code Excerpts	11
4.4	Watermarks	12
4.5	kswapd vs. Direct Reclaim	12
4.6	Pressure Stall Information (PSI)	13
4.7	Hook Justification	13
5	Design and Alternative Methodologies	13
6	Implementation	14
7	Correctness and Safety	14
8	Evaluation	14
9	Conclusion	14

1. Introduction

When physical memory is exhausted, allocating threads can enter **direct reclaim**: they block while the kernel evicts pages synchronously. Under coordinated memory thrash, many threads reclaim in parallel, evicting pages that are immediately refaulted. The working set exceeds available RAM, swap I/O rises, and tail latency degrades.

PressurePause is a kernel modification that adds a bounded, PSI-aware coordination step on the direct reclaim path. When memory PSI full indicates a system-wide stall window, the patch will wake kswapd explicitly and apply a short, capped wait for background progress before falling through to normal direct reclaim. The design never disables reclaim permanently and always preserves stock behavior on timeout or when no reclaimable pages exist.

This report documents the development environment, baseline kernel build, and—in Section 4—identification of the target kernel component within Linux v6.8.12. Later sections reserve space for design, implementation, evaluation, and conclusions as the patched kernel and benchmarks are completed.

2. Environment

- **Host:** Ubuntu 22.04 (Jammy), x86_64
- **Fallback kernel:** 6.8.0-117-generic (distribution build)
- **Custom baseline:** 6.8.12-baseline (vanilla stable v6.8.12, unpatched)
- **Source:** kernel/linux cloned from kernel.org stable tree, tag v6.8.12 (recorded in docs/kernel-tag.txt)
- **Configuration:** `make defconfig` (saved as `kernel/configs/config-6.8.12-baseline`)
- **Build:** `make -j2 LOCALVERSION=-baseline`
- **Tools:** GCC toolchain, dwarves, cscope, stress-ng, linux-tools-\$(uname -r)

Defconfig was chosen instead of copying the Ubuntu `/boot/config-*` file to avoid missing certificate paths (e.g. `SYSTEM_TRUSTED_KEYS`) that broke an earlier vanilla build attempt.

3. Kernel Build and Boot

This section documents the steps to obtain a bootable custom baseline kernel. Terminal screenshots are included for each major step.

3.1 Host Preparation

Build dependencies were installed with apt:

Install build dependencies

```
sudo apt update
sudo apt install -y build-essential libncurses-dev bison flex libssl-dev \
  libelf-dev bc dwarves git cscope stress-ng linux-tools-common \
  linux-tools-$(uname -r)
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$ sudo apt update
Add to Chat Ctrl+L
sudo apt install -y build-essential libncurses-dev bison flex libssl-dev \
  libelf-dev bc dwarves git cscope stress-ng linux-tools-common \
  linux-tools-$(uname -r)
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 https://downloads.cursor.com/aptrepo stable InRelease
Hit:5 http://us.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bc is already the newest version (1.07.1-3build1).
bison is already the newest version (2:3.8.2+dfsg-1build1).
build-essential is already the newest version (12.9ubuntu3).
flex is already the newest version (2.6.4-8build2).
cscope is already the newest version (15.9-1).
git is already the newest version (1:2.34.1-1ubuntu1.17).
libelf-dev is already the newest version (0.186-1ubuntu0.1).
libncurses-dev is already the newest version (6.3-2ubuntu0.1).
libssl-dev is already the newest version (3.0.2-0ubuntu1.23).
linux-tools-6.8.0-117-generic is already the newest version (6.8.0-117.117~22.04.1).
linux-tools-common is already the newest version (5.15.0-179.189).
dwarves is already the newest version (1.25-0ubuntu1~22.04.2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$
```

Figure 1. Installing kernel build and benchmark packages; all packages were already present on the host.

3.2 Source Tree

The stable kernel was cloned shallowly on branch v6.8, then checked out at tag v6.8.12:

Clone and pin kernel tag

```
cd /home/huzaifa/EDU-OS4-KILL-LOOP
mkdir -p kernel docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 \
  https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
```

```

huzaiifa@huzaiifa:~/EDU-OS4-KILL-LOOP$ cd /home/huzaiifa/EDU-OS4-KILL-LOOP
mkdir -p kernel/docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
Cloning into 'kernel/linux'...
remote: Enumerating objects: 88457, done.
remote: Counting objects: 100% (88457/88457), done.
remote: Compressing objects: 100% (86182/86182), done.
remote: Total 88457 (delta 6727), reused 18232 (delta 1354), pack-reused 0 (from 0)
Receiving objects: 100% (88457/88457), 246.16 MiB | 3.12 MiB/s, done.
Resolving deltas: 100% (6727/6727), done.
Note: switching to 'e8f897f4afef0031fe618a8e94127a0934896aba'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

Updating files: 100% (83475/83475), done.
remote: Enumerating objects: 7498, done.
remote: Counting objects: 100% (7498/7498), done.
remote: Compressing objects: 100% (2651/2651), done.
remote: Total 3775 (delta 3636), reused 1226 (delta 1117), pack-reused 0 (from 0)
Receiving objects: 100% (3775/3775), 852.39 KiB | 3.17 MiB/s, done.
Resolving deltas: 100% (3636/3636), completed with 3625 local objects.
From https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux
 * [new tag]          v6.8.12      -> v6.8.12
Previous HEAD position was e8f897f4a Linux 6.8
HEAD is now at 632428373 Linux 6.8.12
huzaiifa@huzaiifa:~/EDU-OS4-KILL-LOOP/kernel/linux$

```

Figure 2. Cloning the stable tree and checking out tag v6.8.12 (HEAD at commit for Linux 6.8.12).

3.3 Kernel Configuration

A default x86_64 configuration was generated and saved for reproducibility:

```

defconfig
cd kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline

```

```
● huzaiifa@huzaiifa:~/EDU-0S4-KILL-LOOP/kernel/linux$ cd /home/huzaiifa/EDU-0S4-KILL-LOOP/kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
*** Default configuration is based on 'x86_64_defconfig'
#
# configuration written to .config
#
#
# No change to .config
#
○ huzaiifa@huzaiifa:~/EDU-0S4-KILL-LOOP/kernel/linux$
```

Figure 3. *make defconfig* based on *x86_64_defconfig*; configuration written to *.config* and copied to *kernel/configs/*.

3.4 Compilation

The kernel was built with a distinct `LOCALVERSION` suffix so it does not collide with distribution packages:

Build baseline kernel

```
make -j2 LOCALVERSION=-baseline 2>&1 | tee ~/build.log
```

```
CC      arch/x86/boot/version.o
CC      arch/x86/boot/video-vga.o
CC      arch/x86/boot/video-vesa.o
CC      arch/x86/boot/video-bios.o
HOSTCC  arch/x86/boot/tools/build
CC      arch/x86/boot/compressed/error.o
CPUSTR  arch/x86/boot/cpustr.h
CC      arch/x86/boot/cpu.o
OBJCOPY arch/x86/boot/compressed/vmlinux.bin
HOSTCC  arch/x86/boot/compressed/mkpiggy
CC      arch/x86/boot/compressed/cpuflags.o
CC      arch/x86/boot/compressed/early_serial_console.o
CC      arch/x86/boot/compressed/kaslr.o
CC      arch/x86/boot/compressed/ident_map_64.o
CC      arch/x86/boot/compressed/idt_64.o
AS      arch/x86/boot/compressed/idt_handlers_64.o
CC      arch/x86/boot/compressed/pgtable_64.o
CC      arch/x86/boot/compressed/acpi.o
CC      arch/x86/boot/compressed/efi.o
AS      arch/x86/boot/compressed/efi_mixed.o
CC      arch/x86/boot/compressed/misc.o
GZIP    arch/x86/boot/compressed/vmlinux.bin.gz
MKPIGGY arch/x86/boot/compressed/piggy.S
AS      arch/x86/boot/compressed/piggy.o
LD      arch/x86/boot/compressed/vmlinux
ZOFFSET arch/x86/boot/zoffset.h
OBJCOPY arch/x86/boot/vmlinux.bin
AS      arch/x86/boot/header.o
LD      arch/x86/boot/setup.elf
OBJCOPY arch/x86/boot/setup.bin
BUILD  arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready  (#1)
huzaiifa@huzaiifa:~/EDU-0S4-KILL-LOOP/kernel/linux$
```

Figure 4. Successful build: `arch/x86/boot/bzImage` is ready for installation.

3.5 Installation

After a successful make, modules and the boot image were installed:

```
Install kernel
sudo make modules_install install
```

```

● huzaifa@huzaifa:~/EDU-054-KILL-LOOP/kernel/linux$ sudo make modules_install install
sudo update-grub
ls /boot/vmlinuz*
[sudo] password for huzaifa:
SYMLINK /lib/modules/6.8.12-baseline/build
INSTALL /lib/modules/6.8.12-baseline/modules.order
INSTALL /lib/modules/6.8.12-baseline/modules.builtin
INSTALL /lib/modules/6.8.12-baseline/modules.builtin.modinfo
INSTALL /lib/modules/6.8.12-baseline/kernel/fs/efivarfs/efivarfs.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/nf_log_syslog.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_mark.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_nat.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_LOG.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_MASQUERADE.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_addrtype.ko
INSTALL /lib/modules/6.8.12-baseline/kernel/net/ipv4/netfilter/iptables_nat.ko
DEPMOD /lib/modules/6.8.12-baseline
INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
update-initramfs: Generating /boot/initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
I: /boot/initrd.img.old is now a symlink to initrd.img-6.8.0-117-generic
I: /boot/initrd.img is now a symlink to initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-shim 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.12-baseline
Found initrd image: /boot/initrd.img-6.8.12-baseline
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done

```

Figure 5. *modules_install* and *install* placing *vmlinuz-6.8.12-baseline* and *modules* under */lib/modules/6.8.12-baseline/*.

3.6 Bootloader Update

GRUB was regenerated so the new kernel appears in the boot menu:

Update GRUB

```

sudo update-grub
ls /boot/vmlinuz*

```

```
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.12-baseline
Found initrd image: /boot/initrd.img-6.8.12-baseline
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
/boot/vmlinuz                /boot/vmlinuz-6.8.12-baseline
/boot/vmlinuz-6.8.0-117-generic /boot/vmlinuz.old
/boot/vmlinuz-6.8.0-40-generic
huzaiifa@huzaiifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 6. *update-grub* detected `/boot/vmlinuz-6.8.12-baseline` alongside Ubuntu `6.8.0-117-generic` and `6.8.0-40-generic`.

3.7 Boot Verification

The GRUB boot menu now lists the custom baseline kernel as the first selectable entry, while the stock Ubuntu kernels remain available as fallback options.

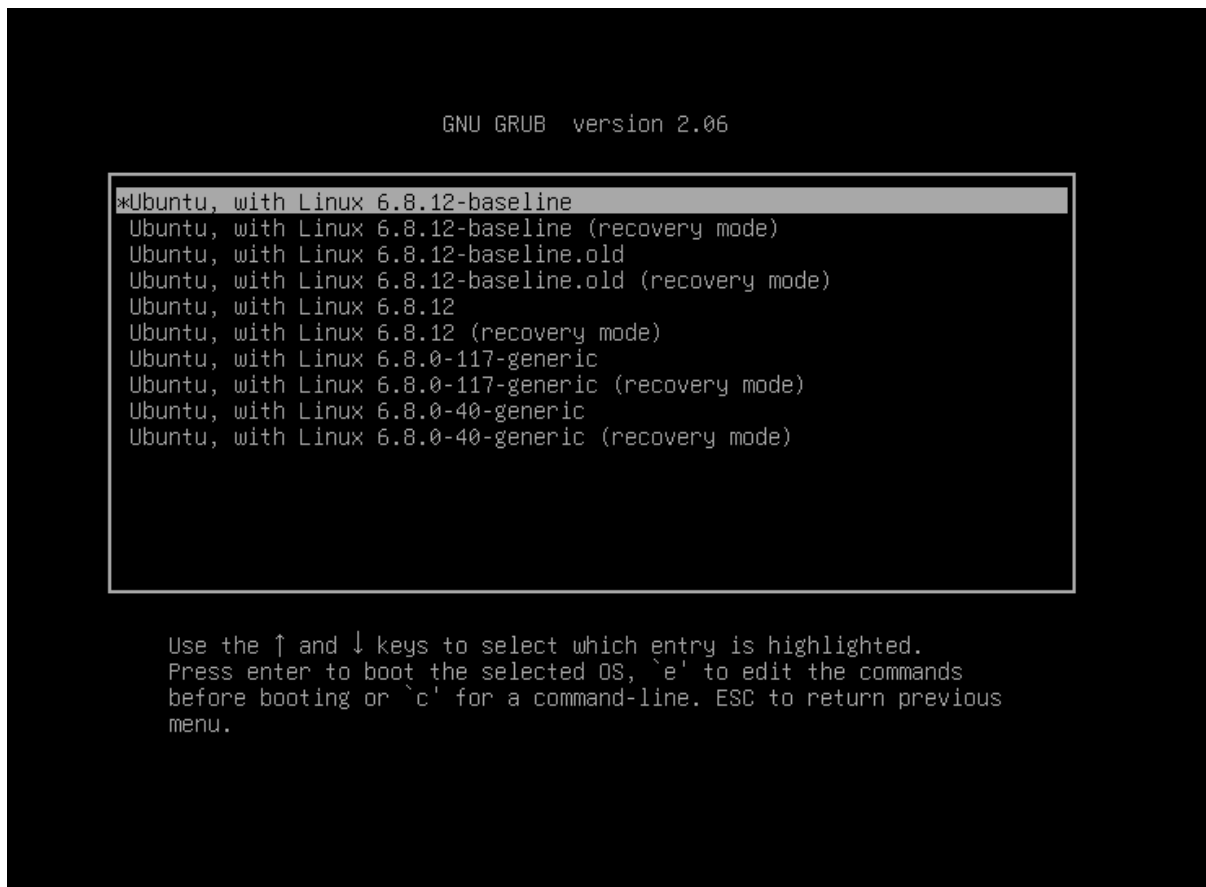


Figure 7. GRUB menu showing *Ubuntu, with Linux 6.8.12-baseline* selected, with recovery and fallback Ubuntu kernel entries still available.

Pending: `uname -r` output confirming 6.8.12-baseline after reboot is not yet included. Boot verification should be completed before the final submission; the fallback Ubuntu entry remains available if the custom kernel fails.

4. Component Identification

This section satisfies the course requirement to identify the selected kernel component within the source tree. The analysis targets Linux **v6.8.12** in `kernel/linux`. A markdown companion is maintained at `docs/identification.md`.

4.1 Selected Component

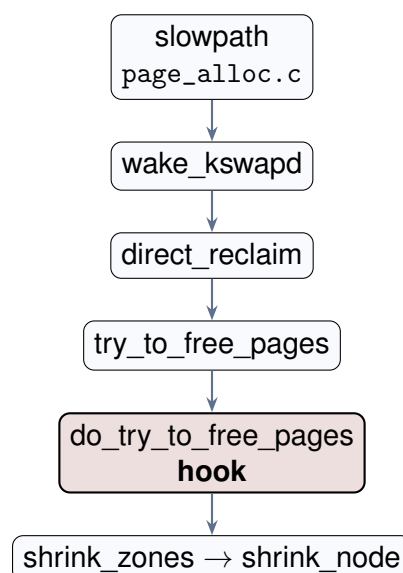
- **Course category:** Page replacement / reclaim (PressurePause extends direct reclaim coordination).
- **Primary files:** `mm/page_alloc.c` (allocation slowpath), `mm/vmscan.c` (reclaim engine).
- **Supporting context:** `kernel/sched/psi.c`, `include/linux/psi_types.h` (PSI accounting).

- **Planned hook (Phase 3):** `do_try_to_free_pages()` in `mm/vmscan.c`, line 6188, immediately before the priority loop that calls `shrink_zones()`.

4.2 Call Graph: Allocation to Reclaim

When `get_page_from_freelist()` cannot satisfy an allocation (zone free pages below watermarks), control enters the allocator slowpath and eventually direct reclaim:

1. `__alloc_pages_slowpath` — `mm/page_alloc.c` ~4040
2. `wake_all_kswapds` — `mm/page_alloc.c` ~3814 (background reclaim hint)
3. `__alloc_pages_direct_reclaim` — `mm/page_alloc.c` ~3781
4. `psi_memstall_enter` / `__perform_reclaim` — `mm/page_alloc.c` ~3755–3790
5. `try_to_free_pages` — `mm/vmscan.c` ~6406
6. `do_try_to_free_pages` — `mm/vmscan.c` ~6188 **[planned hook]**
7. `shrink_zones` → `shrink_node` — `mm/vmscan.c` ~6065 / ~5881



4.3 Key Code Excerpts

Direct reclaim entry from the allocator (`mm/page_alloc.c`):

`mm/page_alloc.c: __perform_reclaim` (lines 3753–3770)

```

1  /* Perform direct synchronous page reclaim */
2  static unsigned long
3  __perform_reclaim(gfp_t gfp_mask, unsigned int order,
4                   const struct alloc_context *ac)
5  {
6      ...
7      progress = try_to_free_pages(ac->zonelist, order, gfp_mask,
8                                  ac->nodemask);
9      ...

```

```

10     return progress;
11 }

```

Public direct-reclaim wrapper (mm/vmscan.c):

mm/vmscan.c: try_to_free_pages (lines 6406–6446)

```

1  unsigned long try_to_free_pages(struct zonelist *zonelist, int order,
2                                gfp_t gfp_mask, nodemask_t *nodemask)
3  {
4      struct scan_control sc = { ... };
5      ...
6      trace_mm_vmscan_direct_reclaim_begin(order, sc.gfp_mask);
7      nr_reclaimed = do_try_to_free_pages(zonelist, &sc);
8      trace_mm_vmscan_direct_reclaim_end(nr_reclaimed);
9      ...
10     return nr_reclaimed;
11 }

```

Reclaim loop and planned hook site (mm/vmscan.c):

mm/vmscan.c: do_try_to_free_pages (lines 6188–6206)

```

1  static unsigned long do_try_to_free_pages(struct zonelist *zonelist,
2                                            struct scan_control *sc)
3  {
4      ...
5      do {
6          sc->nr_scanned = 0;
7          shrink_zones(zonelist, sc); /* LRU eviction per zone */
8          if (sc->nr_reclaimed >= sc->nr_to_reclaim)
9              break;
10         ...
11     } while (--sc->priority >= 0);
12     ...
13 }

```

4.4 Watermarks

Reclaim is triggered when a zone’s free page count falls below configured **min**, **low**, or **high** watermarks during allocation. The fast path in `get_page_from_freelist()` fails; `__alloc_pages_slowpath()` may wake `kswapd`, attempt compaction, and finally call `__alloc_pages_direct_reclaim()` so the allocating thread performs synchronous reclaim.

4.5 kswapd vs. Direct Reclaim

Both paths converge on `shrink_node()`, but they differ in caller context and entry points:

Aspect	Direct reclaim	kswapd
Caller	Allocating task (slowpath)	kswapd kernel thread (~7078)
Entry	try_to_free_pages	balance_pgdat → kswapd_shrink_node
Shared core	shrink_node	shrink_node
GFP context	Caller's gfp_mask	GFP_KERNEL
PressurePause	Yes (hook at do_try_to_free_pages)	No (unless explicitly guarded)

Table 1. Comparison of foreground direct reclaim and background *kswapd*.

4.6 Pressure Stall Information (PSI)

PSI accounts time tasks spend stalled on memory, I/O, or CPU. For PressurePause, the relevant signal is memory PSI **full**: periods when stalled tasks exist and no other tasks are making productive forward progress on memory.

- **Types:** PSI_MEM_SOME, PSI_MEM_FULL in `include/linux/psi_types.h`
- **Accounting:** `kernel/sched/psi.c`
- **Userspace:** `/proc/pressure/memory` (some vs. full averages)

`psi_memstall_enter()` in `__alloc_pages_direct_reclaim` marks the *current allocating task* as memory-stalled during reclaim for PSI statistics. That is distinct from reading system-wide full averages, which the patch will use to decide whether to apply bounded coordination before `shrink_zones()`.

4.7 Hook Justification

`shrink_node()` is shared by both *kswapd* and direct reclaim. Hooking only at `do_try_to_free_pages()` limits changes to the allocating-thread path without affecting background reclaim unless explicitly intended. Hooking at `shrink_node()` would require `!current_is_kswapd()` guards and a broader behavioral surface.

Identification screenshots (editor views of `vmscan.c`, `grep` call chains, or `/proc/pressure/memory` under load) may be added in a future revision; this section relies on verified line numbers from the v6.8.12 tree.

5. Design and Alternative Methodologies

Chosen approach: PSI-gated bounded backoff on direct reclaim entry—wake *kswapd*, wait up to `PRESSURE_PAUSE_MAX_MS`, then fall through to stock reclaim on timeout or lack of progress.

Approach	Pros	Cons
A. PSI-gated back-off (chosen)	Uses kernel stall signal; small code surface	Lock-context constraints; must validate no regressions
B. Fixed sleep in reclaim	Simple to implement	Blind to real pressure; may delay recovery
C. User-space only (swappiness, cgroups)	No kernel patch risk	Does not satisfy course kernel-modification requirement

Table 2. *Alternative methodologies considered for coordinating reclaim under thrash.*

Detailed BEFORE/AFTER flow diagrams and constant definitions (PSI_FULL_THRESHOLD, PRESSURE_PAUSE_MAX_MS) will be completed with the Phase 3 patch.

6. Implementation

To be completed after applying the PressurePause patch to `mm/vmscan.c`, building `LOCALVERSION=-pressurepause` and capturing `git format-patch` output in `kernel/patches/`.

7. Correctness and Safety

To be completed. Planned invariants: bounded wait; never skip reclaim entirely; fall through on timeout or no reclaimable pages; GRUB fallback to Ubuntu kernel if the patched build panics.

8. Evaluation

To be completed. Comparative benchmarks will run only on custom kernels: 6.8.12-baseline (before) vs. 6.8.12-pressurepause (after), using `stress-ng`, `vmstat`, `/proc/pressure/memory`, and `/proc/vmstat (pgmajfault)`. Results will be stored under `results/`.

9. Conclusion

The baseline Linux v6.8.12 kernel was built and installed with `LOCALVERSION=-baseline`. The reclaim component was identified on the direct allocation path from `page_alloc.c` through `try_to_free_pages` and `do_try_to_free_pages` in `vmscan.c`, with PSI providing the system-wide stall signal for the planned modification. Remaining work includes boot verification screenshots, patch implementation, patched-kernel benchmarks, and final results discussion.